Лаб.работа - GestureDemo

Как обычно в Android Studio можно создать стандартный проект из *Empty Activity*.

В файле *MainActivity.kt* удалить функцию Greeting и добавить composable функцию с именем MainScreen:

```
@Composable
fun MainScreen(modifier: Modifier = Modifier) {

}
```

Далее отредактируйте метод *OnCreate()* и функцию GreetingPreview, чтобы они вызывали MainScreen вместо Greeting.

3 Распознавание кликов жестов

Жесты касание (click), также известные как тапы (tap), можно обнаружить на любом видимом компоненте с помощью модификатора *clickable*. Этот модификатор принимает лямбда-выражение, содержащее код, который будет выполняться при нажатии на компонент, к которому он применен, например:

```
SomeComposable(
   modifier = Modifier.clickable { /* Code to be executed */ }
)
```

Отредактируйте файл *MainActivity.kt*, добавьте новый компонент с именем ClickDemo и вызовите его из функции MainScreen:

```
@Composable
fun MainScreen(modifier: Modifier = Modifier) {
   ClickDemo(modifier)
}

@Composable
fun ClickDemo(modifier: Modifier = Modifier) {
   var colorState by remember { mutableStateOf(true)}
   var bgColor by remember { mutableStateOf(Color.Blue) }

   val clickHandler = {

      colorState = !colorState
      bgColor = if (colorState) {
         Color.Blue
      } else {
         Color.DarkGray
      }
   }

   Box(
      modifier
         .clickable { clickHandler() }
         .background(bgColor)
         .size(100.dp)
   )
}
```

Компонуемый элемент ClickDemo содержит компонент Box, цвет фона которого регулируется состоянием *bgColor*. К Box также применен модификатор clickable, настроенный на вызов clickHandler, который, в свою очередь, переключает текущее значение *colorState* и

использует его для переключения текущего значения bgColor между синим и серым. Используйте панель предварительного просмотра в интерактивном режиме, чтобы проверить, что при нажатии на Box цвет фона меняется.

4 Обнаружение нажатий с помощью PointerInputScope

Хотя модификатор clickable полезен для распознавания простых жестов, он не может отличить касания, нажатия, длительные нажатия и двойные касания. Для такого уровня точности нам нужно использовать функцию *detectTapGestures()* в PointerInputScope. Она применяется к composable элементу с помощью модификатора *pointerInput()*, который предоставляет нам доступ к PointerInputScope следующим образом:

```
SomeComposable(
  Modifier
    .pointerInput(Unit) {
      detectTapGestures(
        onPress = { /* Press Detected */ },
        onDoubleTap = { /* Double Tap Detected */ },
        onLongPress = { /* Long Press Detected */ },
        onTap = { /* Tap Detected */ }
      )
    }
)
```

Отредактируйте файл *MainActivity.kt* следующим образом, чтобы добавить и вызвать компонуемый элемент с именем TapPressDemo:

```
@Composable
fun MainScreen(modifier: Modifier = Modifier) {
  TapPressDemo(modifier)
}

@Composable
fun TapPressDemo(modifier: Modifier = Modifier) {

  var textState by remember {mutableStateOf("Waiting ....")}

  val tapHandler = { status : String ->
    textState = status
  }
  Column(
    horizontalAlignment = Alignment.CenterHorizontally,
    modifier = modifier.fillMaxSize()
  ) {
    Box(
      Modifier
        .padding(10.dp)
        .background(Color.Blue)
        .size(100.dp)
        .pointerInput(Unit) {
          detectTapGestures(
            onPress = { tapHandler("onPress Detected") },
            onDoubleTap = { tapHandler("onDoubleTap Detected") },
            onLongPress = { tapHandler("onLongPress Detected") },
            onTap = { tapHandler("onTap Detected") }
```

```
            )
        }
    )
    Spacer(Modifier.height(10.dp))
    Text(textState)
  }
}
```

Компонент TapPressDemo содержит компоненты Box и Text в родительском элементе Column. Строка, отображаемая в компоненте Text, основана на текущем значении состояния *textState*. Когда жест распознается с помощью функции *detectTapGestures()*, вызывается обработчик tapHandler, которому передается новая строка, описывающая тип распознанного жеста. Эта строка присваивается *textState*, в результате чего она отображается в компоненте Text. Обновите панель предварительного просмотра и используйте интерактивный режим, чтобы поэкспериментировать с различными жестами касания и нажатия. Во время работы пользовательский интерфейс должен соответствовать изображению на рисунке 1:

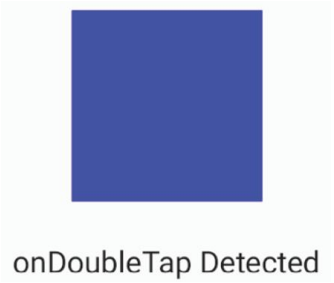


Рис. 1

5 Обнаружение жестов перетаскивания

Мы можем распознавать жесты перетаскивания в компоненте, применив модификатор *draggable()*. Этот модификатор сохраняет смещение (или дельту) движения при перетаскивании от исходной точки по мере его выполнения и сохраняет его в состоянии, экземпляр которого можно создать с помощью вызова функции *rememberDraggableState()*. Затем это состояние можно, например, использовать для перемещения перетаскиваемого компонента в соответствии с жестом. При вызове *draggable()* также необходимо указать, нужно ли отслеживать горизонтальные или вертикальные перемещения.

Чтобы увидеть модификатор *draggable()* в действии, внесите следующие изменения в файл *MainActivity.kt*:

```
@Composable
fun MainScreen(modifier: Modifier = Modifier) {
   DragDemo(modifier)
}

@Composable
fun DragDemo(modifier: Modifier = Modifier) {

   Box(modifier = modifier.fillMaxSize()) {

      var xOffset by remember { mutableStateOf(0f) }

      Box(
         modifier = Modifier
            .offset { IntOffset(xOffset.roundToInt(), 0) }
            .size(100.dp)
            .background(Color.Blue)
            .draggable(
               orientation = Orientation.Horizontal,
               state = rememberDraggableState { distance ->
                  xOffset += distance
               }
            )
```

```
    )
  }
}
```

В примере создаётся состояние для хранения текущего смещения по оси X, которое используется в качестве координаты X для перетаскиваемого блока:

```
var xOffset by remember { mutableStateOf(0f) }

Box(
  modifier = Modifier
    .offset { IntOffset(xOffset.roundToInt(), 0) }
```

Затем к блоку применяется модификатор draggable с параметром ориентации horizontal. Параметр state устанавливается с помощью вызова функции *rememberDraggableState()*, замыкающая лямбда которой используется для получения текущего значения дельты и добавления его к состоянию xOffset. Это, в свою очередь, приводит к перемещению блока в направлении жеста перетаскивания:

```
.draggable(
  orientation = Orientation.Horizontal,
  state = rememberDraggableState { distance ->
    xOffset += distance
  }
)
```

Просмотрите дизайн и убедитесь, что блок можно перетаскивать влево и вправо по горизонтали:



Рис. 2

Модификатор *draggable()* полезен только для поддержки жестов перетаскивания в горизонтальной или вертикальной плоскости. Для поддержки операций перетаскивания в нескольких направлениях необходимо использовать функцию *detectDragGestures* в PointerInputScope.

6 Распознавание жестов перетаскивания с помощью PointerInputScope

Функция *detectDragGestures* в PointerInputScope позволяет одновременно выполнять операции горизонтального и вертикального перетаскивания. Её можно реализовать с помощью следующего синтаксиса:

```
SomeComposable() {
  Modifier
    .pointerInput(Unit) {
      detectDragGestures { _, distance ->
        xOffset += distance.x
        yOffset += distance.y
      }
    }
```

Чтобы увидеть это в действии, добавьте и вызовите новую функцию с именем PointerInputDrag в файле *MainActivity.kt* следующим образом:

```
@Composable
fun MainScreen(modifier: Modifier = Modifier) {
```

```
    PointerInputDrag(modifier)
}

@Composable
fun PointerInputDrag(modifier: Modifier = Modifier) {

    Box(modifier = modifier.fillMaxSize()) {

        var xOffset by remember { mutableStateOf(0f) }
        var yOffset by remember { mutableStateOf(0f) }

        Box(
            Modifier
                .offset { IntOffset(xOffset.roundToInt(), yOffset.roundToInt()) }
                .background(Color.Blue)
                .size(100.dp)
                .pointerInput(Unit) {
                    detectDragGestures { _, distance ->
                        xOffset += distance.x
                        yOffset += distance.y
                    }
                }
        )
    }
}
```

Поскольку мы поддерживаем как горизонтальное, так и вертикальное перетаскивание, мы объявили состояния для хранения смещений по осям X и Y. Лямбда-функция detectDragGestures передает нам объект Offset, который мы назвали *distance* и из которого мы можем получить последние значения смещений по осям X и Y. Они добавляются в состояния *xOffset* и *yOffset* соответственно, благодаря чему компонент Box перемещается по экрану вместе с перетаскиванием:

```
.pointerInput(Unit) {
    detectDragGestures { _, distance ->
        xOffset += distance.x
        yOffset += distance.y
    }
}
```

Просмотрите дизайн в интерактивном режиме и убедитесь, что блок можно перетаскивать в любом направлении на экране:



Рис. 3

7 Прокрутка с помощью модификатора scrollable

На одной из предыдущих лекций мы рассматривали прокрутку, в контексте прокрутки списков элементов. С помощью модификатора *scrollable()* жесты прокрутки не ограничиваются компонентами списка. Как и в случае с модификатором *draggable()*, *scrollable()* поддерживает только горизонтальные или вертикальные жесты, но не оба сразу в одном объявлении модификатора. Управление состоянием прокрутки осуществляется с помощью функции *rememberScrollableState()*, лямбда-выражение которой дает нам доступ к расстоянию, пройденному при прокрутке, которое, в свою очередь, можно использовать для настройки смещения одного или нескольких элементов в иерархии. Внесите следующие изменения, чтобы реализовать прокрутку в файле *MainActivity.kt*:

```
@Composable
fun ScrollableModifier(modifier: Modifier = Modifier) {

  var offset by remember { mutableStateOf(0f) }

  Box(
    modifier
      .fillMaxSize()
      .scrollable(
        orientation = Orientation.Vertical,
        state = rememberScrollableState { distance ->
          offset += distance
          distance
        }
      )
  ) {
    Box(modifier = Modifier
      .size(90.dp)
      .offset { IntOffset(0, offset.roundToInt()) }
      .background(Color.Red))
  }
}
```

Просмотрите новое красное поле и нажмите, а затем перетащите его вертикально по экрану. При этом, что красное поле прокручивается вверх и вниз в ответ на жесты вертикальной прокрутки.

8 Прокрутка с помощью модификаторов прокрутки

Как мы видели в предыдущем примере, модификатор *scrollable()* может определять прокрутку только в одной ориентации. Чтобы определить горизонтальную и вертикальную прокрутку, нам нужно использовать модификаторы прокрутки. По сути, это два модификатора: *verticalScroll()* и *horizontalScroll()* оба должны получать состояние прокрутки, созданное с помощью вызова функции *rememberScrollState()*, например:

```
SomeComposable(modifier = Modifier
  .verticalScroll(rememberScrollState())
  .horizontalScroll(rememberScrollState())) {
}
```

Помимо поддержки прокрутки в обоих направлениях, функции прокрутки также имеют то преимущество, что они управляют самой прокруткой. Это значит, что нам не нужно писать код для применения новых смещений для реализации прокрутки.

Чтобы продемонстрировать эти модификаторы, мы будем использовать компонент Box, содержащий изображение. Размер компонента Box будет соответствовать «области просмот-

ра», через которую в каждый момент времени видна только часть изображения. Вместо этого мы будем использовать прокрутку, чтобы изображение можно было прокручивать внутри компонента.

Первым делом нужно добавить ресурс изображения в проект. В предыдущих проектах мы использовали диспетчер ресурсов для добавления изображения в ресурсы проекта. Кроме этого, можно также скопировать и вставить файл изображения непосредственно в *папку drawables* в окне инструментов проекта.

Изображение, которое мы будем использовать в проекте, называется *vacation.jpg*. ~~Его можно найти в папке *images* с примером кода, который доступен по следующему URL-адресу: *https://www.payloadbooks.com/product/meerkat-compose/*~~

Найдите изображение в проводнике файловой системы вашей операционной системы, выделите его и скопируйте. Щелкните правой кнопкой мыши по записи *app -> res -> drawable* в окне инструментов проекта и выберите в появившемся меню пункт «Paste», чтобы добавить файл в папку:

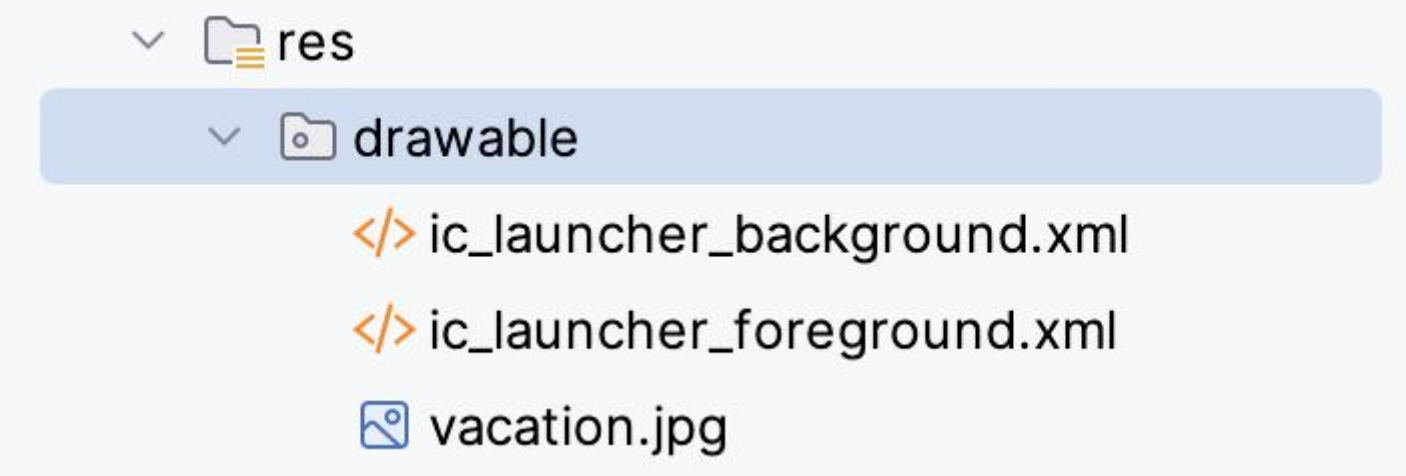


Рис. 4

Далее измените файл *MainActivity.kt* следующим образом:

```
@Composable
fun MainScreen(modifier: Modifier = Modifier) {
   ScrollModifiers(modifier)
}

@Composable
fun ScrollModifiers(modifier: Modifier = Modifier) {

   val image = ImageBitmap.imageResource(id = R.drawable.vacation)

   Box(modifier = modifier
      .size(150.dp)
      .verticalScroll(rememberScrollState())
      .horizontalScroll(rememberScrollState())) {
      Canvas(
         modifier = Modifier
            .size(360.dp, 270.dp)
      )
      {
         drawImage(
            image = image,
            topLeft = Offset(
```

```
                x = 0f,
                y = 0f
            ),
        )
    }
  }
}
```

При предварительном просмотре в интерактивном режиме в компоненте Box будет видна только часть изображения. Если нажать на изображение и перетащить его, можно переместить фотографию так, чтобы были видны другие области изображения:



Рисунок 5

9 Обнаружение жестов сжатия

Наконец рассмотрим жесты, для которых требуется несколько точек касания на экране, начиная с жестов масштабирования (pinch). Жесты масштабирования обычно используются для изменения размера (масштаба) содержимого и создания эффекта увеличения и уменьшения. Этот тип жестов распознаётся с помощью модификатора *transformable(),* который принимает в качестве параметров состояние TransformableState, экземпляр которого можно создать с помощью вызова функции *rememberTransformableState()*. Эта функция принимает замыкающую лямбду, которой передаются следующие три параметра:

- **Изменение масштаба (Scale change)** – значение с плавающей запятой, обновляемое при выполнении жестов масштабирования.
- **Изменение смещения (Offset change)** – экземпляр Offset, содержащий текущие значения смещения по осям X и Y. Это значение обновляется, когда жест приводит к перемещению целевого компонента (называемого *перемещениями – translations*).
- **Изменение поворота (Rotation change)** – значение типа Float, представляющее текущее изменение угла при распознавании жестов поворота.

Все три этих параметра должны быть объявлены при вызове функции *rememberTransformableState()*, даже если они не используются в теле лямбда-выражения. Типичное объявление TransformableState, отслеживающее изменения масштаба, может выглядеть следующим образом:

```
var scale by remember { mutableStateOf(1f) }
val state = rememberTransformableState { scaleChange, offsetChange, rotationChange ->
  scale *= scaleChange
```

```
}
```
После создания состояния его можно использовать при вызове модификатора *transformable()* в composable следующим образом:
```
SomeComposable(modifier = Modifier
   .transformable(state = state) {
   }
)
```
По мере выполнения жеста масштабирования состояние масштабирования будет обновляться. Чтобы отразить эти изменения, нам нужно убедиться, что размер composable элемента также меняется. Мы можем сделать это, обратившись к графическому слою composable элемента и установив для свойств scaleX и scaleY текущее состояние масштабирования. Как мы покажем позже, для преобразований поворота и перемещения также потребуется доступ к графическому слою.

Внесите следующих изменений в файл *MainActivity.kt*, чтобы реализовать распознавание жеста «pinch»:
```
@Composable
fun MainScreen(modifier: Modifier = Modifier) {
   MultiTouchDemo(modifier)
}

@Composable
fun MultiTouchDemo(modifier: Modifier = Modifier) {

   var scale by remember { mutableStateOf(1f) }

   val state = rememberTransformableState {
            scaleChange, offsetChange, rotationChange ->
      scale *= scaleChange
   }

   Box(contentAlignment = Alignment.Center, modifier = modifier.fillMaxSize()) {
      Box(
         Modifier
            .graphicsLayer(
               scaleX = scale,
               scaleY = scale
            )
            .transformable(state = state)
            .background(Color.Blue)
            .size(100.dp)
      )
   }
}
```
Чтобы протестировать жест масштабирования, приложение нужно запустить на устройстве или в эмуляторе, поскольку панель предварительного просмотра пока не поддерживает мультитач-жесты. После запуска выполните жест масштабирования в синем поле, чтобы увеличить или уменьшить масштаб. Если вы используете эмулятор, удерживайте клавишу Ctrl (Cmd на macOS) во время нажатия и перетаскивания, чтобы имитировать множественные касания.

10 Обнаружение жестов вращения

Теперь мы можем добавить в пример поддержку вращения всего с помощью трёх дополнительных строк кода:

```
@Composable
fun MultiTouchDemo(modifier: Modifier = Modifier) {

   var scale by remember { mutableStateOf(1f) }
   var angle by remember { mutableStateOf(0f) }

   val state = rememberTransformableState {
         scaleChange, offsetChange, rotationChange ->
      scale *= scaleChange
      angle += rotationChange
   }

   Box(contentAlignment = Alignment.Center, modifier = Modifier.fillMaxSize()) {
      Box(
         modifier
            .graphicsLayer(
               scaleX = scale,
               scaleY = scale,
               rotationZ = angle
            )
            .transformable(state = state)
            .background(Color.Blue)
            .size(100.dp)
      )
   }
}
```

Скомпилируйте и запустите приложение, выполните жесты масштабирования и поворота. Теперь размер и угол наклона блока должны измениться:

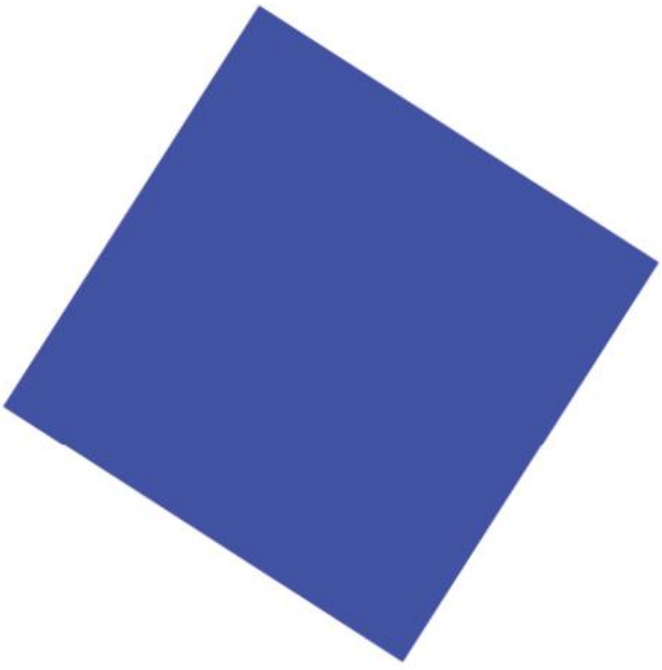


Рис. 6

11 Обнаружение жестов для перемещения (translation)

Перемещение предполагает изменение положения компонента. Как и в случае с определением поворота, мы можем добавить поддержку перемещения в пример, написав всего несколько строк кода:

```
@Composable
fun MultiTouchDemo(modifier: Modifier = Modifier) {

   var scale by remember { mutableStateOf(1f) }
   var angle by remember { mutableStateOf(0f) }
   var offset by remember { mutableStateOf(Offset.Zero) }

   val state = rememberTransformableState {
            scaleChange, offsetChange, rotationChange ->
      scale *= scaleChange
      angle += rotationChange
      offset += offsetChange
   }

   Box(contentAlignment = Alignment.Center, modifier = modifier.fillMaxSize()) {
      Box(
         modifier
            .graphicsLayer(
               scaleX = scale,
               scaleY = scale,
               rotationZ = angle,
               translationX = offset.x,
               translationY = offset.y
            )
            .transformable(state = state)
            .background(Color.Blue)
            .size(100.dp)
      )
   }
}
```

Обратите внимание, что жест перемещения работает только при тестировании на **физическом устройстве**.

Резюме

Жесты – это ключевая форма взаимодействия между пользователем и приложением, работающим на устройстве Android. С помощью функций распознавания жестов в Compose можно реагировать на различные действия на экране, включая нажатия, длительные нажатия, прокрутку, масштабирование и поворот. Жесты распознаются в Compose путем применения модификаторов к компонуемым элементам и реагирования на изменения состояния.